

```
# 1. Install Antigravity
# Download from antigravity.google/download

# 2. In Antigravity IDE agent panel:
"Create React + Vite + TypeScript project with Tailwind, shadcn/ui,
react-router-dom, zustand, react-hook-form, zod, recharts, lucide-react,
sonner, react-dropzone, TanStack Table, date-fns, clsx, tailwind-merge"

# 3. Create .antigravity/rules (paste the rules file above)

# 4. In agent panel:
"Create types, mock data, and mock API layer for ST-SETU"

# 5. Fork into two parallel sessions:
/fork
# Session A: build applicant portal
# Session B: build admin portal

# 6. Use /planning for complex features
# 7. Use /diff before accepting changes
# 8. Use /rewind if something breaks
```

Would you like me to now write out the **exact** `.antigravity/rules` **file** for your project, or the **first agent prompt** to scaffold the entire project in one go?

 54 web pages

give m prd for my project

 Thought for 1 second 

The user wants a PRD (Product Requirements Document) for their ST-SETU frontend project. Let me create a comprehensive, professional PRD that covers everything — since they're building only the frontend, the PRD should reflect that scope but also describe the full product vision.

Let me structure this properly as a real PRD that could be used with Antigravity agents or handed to a dev team.

Product Requirements Document (PRD)

ST-SETU — Scholarship & Fellowship Tribal Education Unified Platform

Version: 1.0

Date: September 2026

Owner: [Your Team Name]

Event: Smart India Hackathon 2026 — SIH25017

Ministry: Ministry of Tribal Affairs (MoTA)

Scope of this PRD: Frontend-only build (React + TypeScript) with mock API layer

1. Executive Summary

ST-SETU is a unified digital platform for end-to-end administration of Ministry of Tribal Affairs (MoTA) scholarship and fellowship schemes for Scheduled Tribe (ST) students. This PRD covers the **frontend implementation** — the complete user-facing product that will later integrate with a real backend.

The frontend will deliver two role-based portals:

- **Applicant Portal** — for ST students applying to schemes like NFST and NOS
- **Admin Portal** — for MoTA officers to verify, scrutinize, select, and monitor

The frontend will use realistic mock data and a mock API layer, allowing a fully clickable demo without a live backend. Every screen, workflow, and state will be functional.

Primary goal: Demonstrate a production-grade, transparent, AI-assisted scholarship management experience that can be handed to a backend team for integration.

2. Problem Statement

The Ministry of Tribal Affairs runs multiple scholarship and fellowship schemes. Today, the process involves:

- Manual scrutiny of thousands of applications
- Repeated correspondence with applicants for missing documents
- Multiple levels of verification across officers
- Limited real-time visibility into scheme progress
- No unified platform — each scheme runs on separate workflows
- Errors and delays in eligibility verification
- No transparency for applicants on their application status

There is a need for a **secure, transparent, intelligent digital platform** that streamlines the end-to-end administration of these schemes.

3. Product Vision

"One integrated, configurable, AI-assisted platform that takes an ST student from application to disbursement — with transparency for the student and full oversight for the Ministry."

The frontend is the **entire experience layer** of this vision. It must:

1. Feel like a real government digital service — trustworthy, accessible, professional
2. Support both roles with distinct, optimized interfaces

- 3. Surface AI assistance visibly — OCR extraction, confidence scores, deficiency flags, anomaly alerts
- 4. Make the workflow transparent — every applicant can see exactly where their application is
- 5. Make the admin workflow efficient — bulk actions, filters, dashboards

4. Goals & Success Metrics

Product Goals

#	Goal	How we know it's achieved
G1	Applicant can apply end-to-end in under 15 minutes	Time-on-task in usability testing
G2	Officer can review and act on an application in under 2 minutes	Time-on-task with sample data
G3	All workflow states are visible and understandable	Applicant can articulate status without help
G4	AI assistance is visible but never overrides human judgment	Officers see confidence scores, all final decisions are human
G5	Works on mobile and desktop	Responsive at 375px, 768px, 1280px, 1920px

Success Metrics (Demo KPIs)

- **15+ pages** fully functional with mock data
- **2 roles** with role-based routing
- **7-step application form** with validation and draft saving
- **5+ charts** on admin dashboard
- **Lighthouse score** ≥ 90 on Performance, Accessibility, Best Practices
- **Zero console errors** on demo path

Out of Scope (for this build)

- Real backend / database
- Real authentication (mock only)
- Real OCR / LLM calls (simulated)
- Real file storage (localStorage only)
- Payment / disbursement integration

- SMS / Email gateways
- Aadhaar / DigiLocker integration

5. Target Users & Personas

Persona 1 — Priya, ST Applicant

- **Age:** 23
- **Context:** [M.Sc.](#) student in Odisha, applying for NFST fellowship for Ph.D.
- **Tech comfort:** Moderate — uses smartphone daily, but not tech-savvy
- **Needs:** Simple application, clear status, no repeated document uploads
- **Pain points:** Doesn't know why applications get rejected; can't track status; has to call the ministry
- **Success:** Applies once, sees status, gets approved without follow-up

Persona 2 — Shri Rajesh, Verification Officer

- **Age:** 45
- **Context:** MoTA officer, handles 200+ applications per day during peak season
- **Tech comfort:** Moderate — uses government portals daily
- **Needs:** Fast verification, clear deficiency marking, bulk actions, audit trail
- **Pain points:** Manual data entry from PDFs; can't prioritize applications; no clear queue
- **Success:** Clears queue efficiently, correct decisions, no rework

Persona 3 — Dr. Meera, Selection Committee Member

- **Age:** 52
- **Context:** Academic on MoTA committee, reviews shortlisted applications for final merit list
- **Tech comfort:** Low-moderate
- **Needs:** Clear merit list, side-by-side comparison, decision rationale
- **Pain points:** No structured view of applicants; committee spends hours on Excel
- **Success:** Finalizes merit list in one session, defensible decisions

Persona 4 — Smt. Kavita, MoTA Administrator

- **Age:** 40
- **Context:** Oversees all schemes, reports to leadership weekly
- **Tech comfort:** Moderate

- **Needs:** Dashboards, scheme-wise analytics, exportable reports
- **Pain points:** No real-time data; relies on manual reports from officers
- **Success:** Opens dashboard, sees live status, exports report in 2 clicks

6. User Roles & Permissions

Role	Can access	Cannot access
Guest	Landing, Login, Register, Scheme info	Any dashboard, any application
Applicant	Own profile, own applications, browse schemes, apply, view own status, notifications, documents	Admin portal, other applicants' data
Verification Officer	Admin dashboard, applications list, verification queue, application review, raise deficiency, approve/reject	Scheme config, final selection, admin settings
Scrutiny Officer	All of the above + scrutiny queue, mark applications for selection	Final merit list approval
Selection Committee	All of the above + merit list, ranking, final approval	Scheme config
MoTA Admin	Everything — scheme config, dashboards, reports, communications, settings, audit logs	—

For the frontend build, we will implement **two primary roles** (Applicant and Admin) with the admin role having the full set of admin capabilities. Sub-roles can be simulated via a role switcher in the demo.

7. Scope — Feature List

7.1 Public Pages

ID	Feature	Priority	Notes
PUB-1	Landing page	P0	Hero, schemes overview, CTAs, fo

ID	Feature	Priority	Notes
PUB-2	Login page	P0	Role-based redirect after login
PUB-3	Register page	P0	Applicant registration with validation
PUB-4	Forgot password	P1	Simulated flow
PUB-5	About / Help page	P2	Static content

7.2 Applicant Portal

ID	Feature	Priority	Notes
APP-1	Applicant dashboard	P0	KPIs, latest application status, notifications
APP-2	Browse schemes	P0	NFST, NOS, others with eligibility summary
APP-3	Scheme detail	P0	Full eligibility, documents required, timeline
APP-4	Multi-step apply form	P0	7 steps, draft save, validation, specific fields
APP-5	Document upload	P0	Drag-drop, preview, simulate OCR, document extraction
APP-6	My Applications	P0	Table with status chips, filters
APP-7	Application detail	P0	Full view, status timeline, deficiency panel
APP-8	Respond to deficiency	P0	Upload revised documents, re-submit
APP-9	Notifications	P0	In-app notification center
APP-10	Profile management	P1	Edit profile, change password
APP-11	Download acknowledgment	P1	PDF of submitted application

7.3 Admin Portal

ID	Feature	Priority	Notes
ADM-1	Admin dashboard	P0	KPI cards + 5 charts + recent queries
ADM-2	Applications list	P0	TanStack table, filters, search, bulk actions

ID	Feature	Priority	Notes
ADM-3	Application review	P0	Split view: data + OCR-annotate documents
ADM-4	Verification queue	P0	Priority-sorted, confidence score
ADM-5	Deficiency raise	P0	Modal with checklist, sends noti
ADM-6	Scrutiny workflow	P0	Mark verified apps for scrutiny
ADM-7	Selection / Merit list	P0	Rank, reorder, generate merit list
ADM-8	Scheme config engine	P0	Create/edit schemes with dynan
ADM-9	Communications	P1	Template list, bulk send, log
ADM-10	Reports & analytics	P0	Charts, filters, export
ADM-11	Audit log	P1	Every action logged, filterable
ADM-12	Settings	P2	Roles, permissions, profile

7.4 Shared

ID	Feature	Priority
SH-1	Role-based routing	P0
SH-2	Protected routes	P0
SH-3	Toast notifications	P0
SH-4	Loading states	P0
SH-5	Empty states	P0
SH-6	Error states	P0
SH-7	Responsive layouts	P0
SH-8	Dark mode (optional)	P2
SH-9	Multi-language EN/HI (optional)	P2

8. Detailed Feature Specifications

8.1 Multi-Step Application Form (APP-4)

Purpose: Allow applicant to complete a full application across multiple steps with the ability to save and resume.

Steps:

1. **Personal Details** — Name, DOB, gender, category (ST auto-selected), Aadhaar (masked), phone, email
2. **Address & Domicile** — State, district, PIN, permanent address, domicile certificate reference
3. **Academic Details** — Highest qualification, institution, course, year, marks/percentage
4. **Scheme-Specific** — Fields change based on scheme:
 - NFST: Research topic, guide name, university, Ph.D. registration number
 - NOS: Foreign university name, country, program, admission letter reference, GRE/TOEFL scores
5. **Bank Details** — Account number, IFSC, bank name, branch, passbook upload
6. **Documents Upload** — Caste certificate, income certificate, marksheets, admission letter, passport (NOS only)
7. **Review & Submit** — Read-only summary with edit links, declaration checkbox, submit

Behavior:

- Progress stepper visible at top
- "Save as Draft" available at every step (auto-saves to localStorage on change)
- Validation with react-hook-form + zod, per step
- Cannot proceed if current step has errors
- On submit, generate application ID like NFST/2026/00123
- Confirmation screen with download acknowledgment PDF

Acceptance Criteria:

- Draft saved and restored across browser refresh
- All fields validated with India-specific rules (PIN code 6 digits, IFSC format, Aadhaar 12 digits, phone 10 digits)
- Scheme-specific fields render correctly based on `schemeId` param
- Review step shows all entered data in read-only cards

8.2 Application Review with OCR Preview (ADM-3)

Purpose: Allow officer to verify an application against uploaded documents with AI-extracted data visible.

Layout:

- **Left panel (40%):** Applicant data in read-only cards
 - Personal, Address, Academic, Bank, Scheme-specific
 - Each field has a small "verified" checkbox (officer confirms)
- **Right panel (60%):** Document viewer
 - Tabs for each uploaded document
 - PDF/image preview
 - Overlay: extracted fields with confidence bars
 - Fields with confidence < 70% highlighted red
 - Click on a field to jump to the source location in the document

Actions (bottom bar):

-  **Approve** — marks application as verified
-  **Reject** — opens modal with reason (from predefined list + free text)
-  **Raise Deficiency** — opens modal with checklist:
 - Missing document
 - Illegible document
 - Data mismatch
 - Expired certificate
 - Other (free text)
 - On submit, sends notification to applicant

Acceptance Criteria:

- Officer can see all data and documents on one screen
- Every action logs to audit trail
- Deficiency modal sends a mock notification visible in applicant portal
- Confidence scores visible on extracted fields
- Keyboard shortcuts: A = approve, R = reject, D = deficiency

8.3 Admin Dashboard (ADM-1)

Purpose: Give MoTA leadership a real-time view of all scheme activity.

KPI Cards (top row):

- Total Applications (with % change from last period)
- Pending Verification
- Verified
- Selected

- Rejected / Deficient

Charts (middle rows):

- **Line chart:** Applications submitted over last 30 days
- **Donut chart:** Scheme-wise split (NFST vs NOS vs others)
- **Bar chart:** State-wise applications (top 10 states)
- **Funnel chart:** Submitted → Verified → Scrutinized → Selected → Disbursed
- **Stacked bar:** Deficiency reasons breakdown

Recent Activity:

- Table of last 10 submissions needing action
- Anomaly alert panel (AI-flagged applications)

Filters:

- Date range
- Scheme
- State
- Status

Acceptance Criteria:

- All charts render correctly with mock data
- Filters update all charts
- Hover tooltips show exact values
- Mobile view stacks charts vertically

8.4 Scheme Configuration Engine (ADM-8)

Purpose: Allow MoTA admin to configure new schemes without code changes.

Fields:

- Scheme name, code, description
- Application window (start/end dates)
- Eligibility rules (dynamic list):
 - Field (e.g., "Category", "Annual Income", "Qualification")
 - Operator (equals, less than, greater than, in list)
 - Value
 - Multiple rules with AND/OR logic
- Required documents (checklist with add/remove)

- Workflow stages (drag-to-reorder list)
- Selection criteria (merit vs need-based, weights)

Preview:

- Right panel shows a live preview of how the applicant form will render with these rules

Acceptance Criteria:

- Admin can add, edit, delete schemes
- Rules are validated (no empty fields)
- Preview updates live
- Saved scheme appears in applicant's Browse Schemes page

8.5 Deficiency Workflow (APP-8 + ADM-5)

Flow:

1. Officer raises deficiency on an application → status changes to `query_raised`
2. Applicant sees notification + red badge on application
3. Applicant opens application → sees deficiency card with list of issues
4. Applicant uploads corrected documents → clicks "Resubmit"
5. Status changes back to `under_verification`
6. Officer sees the resubmitted application in queue with a "Resubmission" badge

Acceptance Criteria:

- End-to-end flow works with mock data
- Notification appears in applicant's notification center
- Status timeline shows the deficiency round
- Multiple deficiency rounds supported (Round 1, Round 2, ...)

9. Information Architecture

text

```

ST-SETU
|
├─ Public
|   ├── Landing (/)
|   ├── Login (/login)
|   ├── Register (/register)
|   └─ Help (/help)
|
├─ Applicant Portal (/app)

```

```

|   ├── Dashboard
|   ├── Schemes
|   |   ├── List
|   |   └── Detail (:schemeId)
|   ├── Apply (:schemeId)           [multi-step form]
|   ├── My Applications
|   |   ├── List
|   |   └── Detail (:applicationId)
|   ├── Documents
|   ├── Notifications
|   └── Profile
|
└── Admin Portal (/admin)
    ├── Dashboard
    ├── Applications
    |   ├── List
    |   └── Review (:applicationId)
    ├── Verification Queue
    ├── Scrutiny
    ├── Selection
    |   └── Merit List
    ├── Schemes
    |   ├── List
    |   └── Configure (:schemeId)
    ├── Communications
    ├── Reports
    ├── Audit Log
    └── Settings

```

10. Data Models (TypeScript Interfaces)

ts

```
export type Role = 'applicant' | 'admin' | 'officer' | 'committee';
```

```
export type ApplicationStatus =
  | 'draft'
  | 'submitted'
  | 'under_verification'
  | 'query_raised'
  | 'verified'
  | 'scrutinized'
  | 'selected'
  | 'rejected'
  | 'disbursed';
```

```
export interface User {
  id: string;
  name: string;
  email: string;
  phone: string;
  role: Role;
  createdAt: string;
}
```

```
}

export interface Scheme {
  id: string;
  code: 'NFST' | 'NOS' | string;
  name: string;
  description: string;
  window: { start: string; end: string };
  eligibility: EligibilityRule[];
  requiredDocs: string[];
  stages: string[];
  selectionCriteria: 'merit' | 'need' | 'hybrid';
}
```

```
export interface EligibilityRule {
  field: string;
  operator: 'eq' | 'lt' | 'gt' | 'in' | 'range';
  value: string | number | string[];
  logic: 'AND' | 'OR';
}
```

```
export interface Application {
  id: string;
  applicantId: string;
  applicantName: string;
  schemeId: string;
  schemeCode: string;
  status: ApplicationStatus;
  currentStage: number;
  submittedAt: string;
  lastUpdatedAt: string;
  personal: PersonalDetails;
  address: AddressDetails;
  academic: AcademicDetails;
  schemeSpecific: Record<string, any>;
  bank: BankDetails;
  documents: Document[];
  ocrFields?: OCRField[];
  deficiency?: Deficiency;
  score?: number;
  remarks?: string;
}
```

```
export interface Document {
  id: string;
  type: string;
  fileName: string;
  uploadedAt: string;
  status: 'pending' | 'verified' | 'deficient';
  url: string; // mock URL
}
```

```
export interface OCRField {
  field: string;
  value: string;
  confidence: number; // 0-100
}
```

```

    sourceDocId: string;
    sourcePage: number;
  }

  export interface Deficiency {
    raisedAt: string;
    raisedBy: string;
    reasons: string[];
    note?: string;
    resolvedAt?: string;
    round: number;
  }

  export interface Notification {
    id: string;
    userId: string;
    title: string;
    message: string;
    type: 'info' | 'success' | 'warning' | 'error';
    read: boolean;
    link?: string;
    createdAt: string;
  }

  export interface AuditLogEntry {
    id: string;
    actorId: string;
    actorName: string;
    action: string;
    entityType: 'application' | 'scheme' | 'user';
    entityId: string;
    timestamp: string;
    metadata?: Record<string, any>;
  }

```

11. Mock API Contract

All frontend pages consume this interface. Later, replace with real `fetch()` calls — no UI rewrites needed.

```

ts

export const mockApi = {
  // Auth
  login(email: string, password: string): Promise<{ user: User; token: string }>;
  register(data: RegisterInput): Promise<User>;
  logout(): Promise<void>;

  // Schemes
  getSchemes(): Promise<Scheme[]>;
  getSchemeById(id: string): Promise<Scheme>;
  createScheme(data: Partial<Scheme>): Promise<Scheme>;
  updateScheme(id: string, data: Partial<Scheme>): Promise<Scheme>;

```

```

// Applications
getApplications(filters?: ApplicationFilters): Promise<Application[]>;
getApplicationById(id: string): Promise<Application>;
getMyApplications(userId: string): Promise<Application[]>;
submitApplication(data: Partial<Application>): Promise<Application>;
updateApplication(id: string, data: Partial<Application>): Promise<Application>;

// Documents
uploadDocument(file: File, applicationId: string): Promise<Document>;
runOCR(documentId: string): Promise<OCRField[]>; // simulates AI extraction

// Workflow
approveApplication(id: string): Promise<Application>;
rejectApplication(id: string, reason: string): Promise<Application>;
raiseDeficiency(id: string, reasons: string[], note?: string): Promise<Application>;
resubmitApplication(id: string, documents: File[]): Promise<Application>;
markForSelection(id: string, score: number): Promise<Application>;
generateMeritlist(schemeId: string): Promise<Application[]>;

// Dashboards
getAdminStats(filters?: StatsFilters): Promise<AdminStats>;
getApplicantStats(userId: string): Promise<ApplicantStats>;

// Notifications
getNotifications(userId: string): Promise<Notification[]>;
markNotificationRead(id: string): Promise<void>;

// Audit
getAuditLog(filters?: AuditFilters): Promise<AuditLogEntry[]>;
};

```

Every method:

- Returns a `Promise`
- Simulates 300–800ms network delay
- Throws errors for invalid input (e.g., wrong credentials)
- Reads/writes to `localStorage` for persistence across refreshes

12. Design System

12.1 Color Palette

Token	Hex	Usage
navy	#1F3864	Headers, primary text, primary buttons
navy-light	#2E508C	Hover states
orange	#ED7D31	Accent, CTAs, active states, highlights

Token	Hex	Usage
green	#4CAF50	Success, verified, selected
red	#C62828	Errors, rejected, deficient
teal	#2A9D8F	Info, secondary accents
purple	#6F42A0	Selection, committee
yellow	#F5C218	Warnings, pending
bg	#F8FAFC	Page background
card	FFFFFF	Cards, panels
border	#E2E8F0	Dividers, borders
text-primary	#1A202C	Body text
text-muted	#718096	Secondary text

12.2 Typography

Style	Font	Size	Weight
Display	Inter	32px	700
H1	Inter	24px	700
H2	Inter	20px	600
H3	Inter	18px	600
Body	Inter	14px	400
Small	Inter	12px	400
Mono	JetBrains Mono	13px	400

12.3 Spacing

Scale: 4 / 8 / 12 / 16 / 20 / 24 / 32 / 40 / 48 / 64

12.4 Components

- **Buttons:** primary (navy), secondary (outline), danger (red), ghost
- **Cards:** 12px radius, `shadow-sm`, `hover shadow-md`
- **Chips:** status badges with role-appropriate colors

- **Inputs:** 8px radius, 1px border, focus ring in orange
- **Stepper:** horizontal with numbered circles
- **Timeline:** vertical with connecting line
- **Table:** zebra rows, sticky header, sortable columns
- **Modal:** centered, backdrop blur, close on Esc
- **Toast:** top-right, 4s auto-dismiss, stackable

13. Layout Guidelines

Applicant Portal

- **Desktop:** Sidebar (240px) + main content (max-width 1200px)
- **Tablet:** Collapsible sidebar
- **Mobile:** Bottom nav with 5 items (Dashboard, Schemes, Apply, Applications, Profile)

Admin Portal

- **Desktop:** Sidebar (260px) + main content (max-width 1440px) — wider for tables
- **Tablet:** Collapsible sidebar
- **Mobile:** Not primary use case — show "Best viewed on desktop" notice

Header

- Navy background bar
- Page title on left
- Notifications bell, profile menu on right

14. Non-Functional Requirements

Performance

- First Contentful Paint < 1.5s
- Time to Interactive < 3s on 4G
- Lighthouse Performance ≥ 90
- Bundle size < 500 KB gzipped

Accessibility

- WCAG 2.1 AA compliant

- Keyboard navigable
- Screen reader tested
- Color contrast $\geq 4.5:1$
- Focus indicators visible
- ARIA labels on all interactive elements

Responsiveness

- Breakpoints: 375px, 640px, 768px, 1024px, 1280px, 1536px
- Mobile-first CSS
- Touch targets ≥ 44 px

Browser Support

- Chrome 90+, Edge 90+, Firefox 88+, Safari 14+
- No IE support

Internationalization (Optional)

- English (default)
- Hindi (i18n-ready with `react-i18next`)

Security (Frontend-Level)

- No sensitive data in localStorage beyond mock scope
- Role-based route guards
- Input sanitization on all forms
- No `dangerouslySetInnerHTML`

15. Tech Stack

Layer	Choice
Framework	React 19 + Vite
Language	TypeScript 5
Styling	Tailwind CSS 4
Components	shadcn/ui + Radix UI
Icons	lucide-react

Layer	Choice
Routing	react-router-dom v7
State	Zustand
Forms	react-hook-form + zod
Tables	TanStack Table v8
Charts	Recharts
Toasts	sonner
File upload	react-dropzone
PDF preview	react-pdf
Dates	date-fns
Utils	clsx + tailwind-merge
Mock API	MSW or plain async functions
Persistence	localStorage
Deployment	Vercel

16. Project Structure

text

```

st-setu-frontend/
├─ public/
│   └─ mock/
├─ src/
│   ├─ components/
│   │   ├─ ui/                # shadcn primitives
│   │   ├─ layout/           # Navbar, Sidebar, Footer, PageHeader
│   │   ├─ applicant/        # Applicant-specific components
│   │   ├─ admin/            # Admin-specific components
│   │   └─ shared/           # DashboardCard, ChartCard, Stepper, EmptyState
│   ├─ pages/
│   │   ├─ Landing.tsx
│   │   ├─ auth/
│   │   ├─ applicant/
│   │   ├─ admin/
│   │   └─ NotFound.tsx
│   ├─ context/
│   │   └─ AuthContext.tsx
│   ├─ hooks/
│   └─ lib/

```

```

| | | └─ mockApi.ts
| | | └─ mockData.ts
| | | └─ utils.ts
| | | └─ validators.ts
| | └─ routes/
| |   └─ AppRoutes.tsx
| | └─ styles/
| | └─ types/
| |   └─ index.ts
| | └─ App.tsx
| |   └─ main.tsx
└─ .antigravity/
  └─ rules
└─ .env
└─ index.html
└─ package.json
└─ tailwind.config.ts
└─ tsconfig.json
└─ README.md

```

17. Build Plan (7 Days)

Day	Deliverable
Day 1	Project setup, .antigravity/rules, design tokens, layout shell (Navbar, Sidebar, Footer)
Day 2	Auth pages, AuthContext, role guards, routing, mock data + mock API base
Day 3	Applicant dashboard, Browse schemes, Scheme detail, My Applications, Application detail
Day 4	Multi-step apply form, Document upload, Deficiency response
Day 5	Admin dashboard with charts, Applications table, Application review with OCR preview
Day 6	Verification queue, Scrutiny, Selection, Merit list, Scheme config, Communications
Day 7	Polish, animations, responsive fixes, accessibility pass, deploy, demo video

18. Demo Script (for SIH judges)

Total: 6 minutes

- 1. **0:00–0:30** — Landing page. Introduce problem and ST-SETU.
- 2. **0:30–1:00** — Register as applicant. Show validation.
- 3. **1:00–2:30** — Browse schemes → Apply for NFST → Complete 7-step form → Upload docs → Submit. Show draft auto-save.
- 4. **2:30–3:00** — Applicant dashboard. Show status timeline and notification.
- 5. **3:00–3:30** — Switch to admin (demo credentials). Show dashboard with live KPIs and charts.
- 6. **3:30–4:30** — Open the same application in review. Show OCR-extracted fields with confidence scores. Raise a deficiency.
- 7. **4:30–5:00** — Switch back to applicant. Show notification and deficiency response flow.
- 8. **5:00–5:30** — Back to admin. Approve. Show status change.
- 9. **5:30–6:00** — Merit list generation. Reports. Wrap up with impact metrics.

19. Risks & Mitigations

Risk	Impact	Mitigation
Scope creep	Missing demo deadline	Strict P0/P1/P2 prioritization. P2 features dropped if needed.
Mock data too thin	Demo feels hollow	Generate 50+ realistic applications with Indian names, states, institutions
OCR simulation unconvincing	AI value unclear	Pre-compute mock OCR results with realistic confidence scores
Mobile view broken	Accessibility concerns	Test at 375px on every page
Theme inconsistency	Looks unprofessional	Design tokens in Tailwind config; no hardcoded colors
Role confusion	Demo breaks	Clear role switcher in dev mode; test both paths daily
Slow build due to agent loops	Missing features	Use Antigravity /planning before each major feature; /fork for parallel work

20. Acceptance Criteria (Definition of Done)

The frontend is **done** when:

- All PO features are implemented and functional
- Both roles (applicant, admin) can complete their full workflow end-to-end
- Application state persists across browser refresh (localStorage)
- All screens are responsive at 375px, 768px, 1280px
- Lighthouse ≥ 90 on Performance, Accessibility, Best Practices
- Zero console errors on demo path
- All forms validated with India-specific rules
- All charts render with mock data and filters work
- Deficiency workflow works end-to-end (raise → notify → respond → re-verify)
- Merit list can be generated and exported
- Deployed to Vercel with a public URL
- Demo video recorded (≤ 6 minutes)
- README includes setup, credentials, and architecture
- `.antigravity/rules` documented and shared with team

21. Reference Links

- MoTA Scholarships: <https://tribal.nic.in/ScholarshiP.aspx>
- DBT Tribal: <https://dbttribal.gov.in/AllScheme.aspx>
- National Scholarship Portal: <https://scholarships.gov.in>
- Digital India: <https://digitalindia.gov.in>
- WCAG 2.1: <https://www.w3.org/WAI/WCAG21/quickref/>
- React 19 Docs: <https://react.dev>
- Tailwind CSS: <https://tailwindcss.com>
- shadcn/ui: <https://ui.shadcn.com>
- Recharts: <https://recharts.org>
- TanStack Table: <https://tanstack.com/table>

22. Appendix — Demo Credentials

Role	Email	Password
Applicant	student@demo.in	demo123
Officer	officer@demo.in	demo123

Role	Email	Password
Admin	admin@demo.in	demo123

23. Appendix — Sample Mock Data Volume

- **3 schemes:** NFST, NOS, and one more (e.g., "Top Class Education")
- **50 applicants** with realistic Indian names, states, universities
- **80 applications** distributed across schemes and statuses:
 - 20 submitted
 - 15 under verification
 - 10 query raised
 - 15 verified
 - 10 scrutinized
 - 8 selected
 - 2 rejected
- **200+ documents** across applications with mock OCR results
- **10 notifications** per applicant on average
- **300+ audit log entries**

End of PRD

Would you like me to next generate:

1. **The exact** `.antigravity/rules` **file** based on this PRD
2. **The first agent prompt** to scaffold the entire project in one go
3. **The mock data JSON files** (schemes, applications, users) ready to paste
4. **The multi-step application form component** written out completely

Tell me which one and I'll produce it.

can you give me this in document form

⊠ Thought for 2 seconds ▾

The user wants the PRD in document form. I should provide it as a downloadable file. I can give them a Markdown file they can save, or I could offer to generate it as a proper .docx or PDF. Let me provide it as a clean Markdown document that they can copy-paste and also give them instructions for converting to DOCX/PDF. Actually, the best approach is to give them a Python

script that generates a proper .docx file, since they can just run it. Let me also give them the markdown version.

Actually let me think about what's most useful. They asked "can you give me this in document form". This could mean:

1. A downloadable file format
2. A clean document structure

The most practical thing is to give them:

1. A Python script to generate a .docx file
2. Or a Markdown file they can convert

Let me provide a Python script using python-docx that generates a professional-looking PRD document. That way they can run it and get a real Word document.

Let me write this out carefully with all the sections from my previous PRD.

PRD Document Generator

Here's a Python script that generates a **professional, formatted** .docx **PRD** for ST-SETU. Run it once and you get a Word document you can open, edit, and share.

How to run

```
bash
```

```
pip install python-docx
python generate_prd.py
```

Output: ST-SETU-PRD.docx — a fully formatted document with title page, table of contents, headings, tables, and code blocks.

generate_prd.py

```
python
```

```
"""
```

```
Generates the ST-SETU PRD as a professional .docx document.
Includes title page, TOC, styled headings, tables, and code blocks.
"""
```

```
from docx import Document
from docx.shared import Pt, Inches, RGBColor, Cm
from docx.enum.text import WD_ALIGN_PARAGRAPH, WD_BREAK
from docx.enum.table import WD_TABLE_ALIGNMENT
from docx.xml.ns import qn
from docx.xml import XmlElement
```

```
# ----- HELPERS
```

```
NAVY = RGBColor(0x1F, 0x38, 0x64)
ORANGE = RGBColor(0xED, 0x7D, 0x31)
```

```
GRAY = RGBColor(0x60, 0x60, 0x60)
DARK = RGBColor(0x22, 0x22, 0x22)
```

```
def set_cell_bg(cell, hex_color):
    tcPr = cell._tc.get_or_add_tcPr()
    shd = XmlElement('w:shd')
    shd.set(qn('w:val'), 'clear')
    shd.set(qn('w:color'), 'auto')
    shd.set(qn('w:fill'), hex_color)
    tcPr.append(shd)
```

```
def add_heading(doc, text, level=1, color=NAVY, size=None, space_before=14):
    p = doc.add_paragraph()
    p.paragraph_format.space_before = Pt(space_before)
    p.paragraph_format.space_after = Pt(6)
    run = p.add_run(text)
    run.bold = True
    run.font.color.rgb = color
    run.font.name = 'Calibri'
    if size is None:
        size = {1: 18, 2: 15, 3: 13, 4: 12}.get(level, 12)
    run.font.size = Pt(size)
    return p
```

```
def add_para(doc, text, size=11, bold=False, italic=False,
             color=DARK, align=WD_ALIGN_PARAGRAPH.LEFT, space_after=6):
    p = doc.add_paragraph()
    p.alignment = align
    p.paragraph_format.space_after = Pt(space_after)
    run = p.add_run(text)
    run.bold = bold
    run.italic = italic
    run.font.size = Pt(size)
    run.font.color.rgb = color
    run.font.name = 'Calibri'
    return p
```

```
def add_bullet(doc, text, size=11, level=0):
    p = doc.add_paragraph(style='List Bullet')
    p.paragraph_format.left_indent = Inches(0.3 + level * 0.25)
    p.paragraph_format.space_after = Pt(3)
    run = p.add_run(text)
    run.font.size = Pt(size)
    run.font.name = 'Calibri'
    run.font.color.rgb = DARK
    return p
```

```
def add_numbered(doc, text, size=11):
    p = doc.add_paragraph(style='List Number')
    p.paragraph_format.left_indent = Inches(0.3)
    p.paragraph_format.space_after = Pt(3)
```

```

run = p.add_run(text)
run.font.size = Pt(size)
run.font.name = 'Calibri'
return p

def add_table(doc, headers, rows, col_widths=None):
    table = doc.add_table(rows=1, cols=len(headers))
    table.style = 'Light Grid Accent 1'
    table.alignment = WD_TABLE_ALIGNMENT.CENTER
    hdr = table.rows[0].cells
    for i, h in enumerate(headers):
        hdr[i].text = ''
        p = hdr[i].paragraphs[0]
        run = p.add_run(h)
        run.bold = True
        run.font.size = Pt(10.5)
        run.font.color.rgb = RGBColor(0xFF, 0xFF, 0xFF)
        run.font.name = 'Calibri'
        set_cell_bg(hdr[i], '1F3864')
    for row in rows:
        cells = table.add_row().cells
        for i, val in enumerate(row):
            cells[i].text = ''
            p = cells[i].paragraphs[0]
            run = p.add_run(str(val))
            run.font.size = Pt(10)
            run.font.name = 'Calibri'
            run.font.color.rgb = DARK
    if col_widths:
        for i, w in enumerate(col_widths):
            for row in table.rows:
                row.cells[i].width = Inches(w)
    doc.add_paragraph()
    return table

def add_code_block(doc, code):
    p = doc.add_paragraph()
    p.paragraph_format.left_indent = Inches(0.2)
    p.paragraph_format.space_before = Pt(6)
    p.paragraph_format.space_after = Pt(6)
    run = p.add_run(code)
    run.font.name = 'Consolas'
    run.font.size = Pt(9)
    run.font.color.rgb = RGBColor(0x1A, 0x1A, 0x1A)
    # shading
    pPr = p._p.get_or_add_pPr()
    shd = XmlElement('w:shd')
    shd.set(qn('w:val'), 'clear')
    shd.set(qn('w:fill'), 'F2F5FA')
    pPr.append(shd)

def page_break(doc):
    doc.add_paragraph().add_run().add_break(WD_BREAK.PAGE)

```

```

# ----- DOCUMENT

doc = Document()

# Set default font
style = doc.styles['Normal']
style.font.name = 'Calibri'
style.font.size = Pt(11)
style.font.color.rgb = DARK

# Narrow margins
for section in doc.sections:
    section.top_margin = Cm(2)
    section.bottom_margin = Cm(2)
    section.left_margin = Cm(2.2)
    section.right_margin = Cm(2.2)

# =====
# TITLE PAGE
# =====

for _ in range(4):
    doc.add_paragraph()

add_para(doc, "PRODUCT REQUIREMENTS DOCUMENT",
        size=14, bold=True, color=ORANGE,
        align=WD_ALIGN_PARAGRAPH.CENTER, space_after=4)

add_para(doc, "ST-SETU",
        size=36, bold=True, color=NAVY,
        align=WD_ALIGN_PARAGRAPH.CENTER, space_after=2)

add_para(doc, "Scholarship & Fellowship Tribal Education Unified Platform",
        size=14, color=GRAY,
        align=WD_ALIGN_PARAGRAPH.CENTER, space_after=20)

add_para(doc, "AI-Enabled Scholarship and Fellowship Management System for Scheduled Tribes",
        size=12, italic=True, color=DARK,
        align=WD_ALIGN_PARAGRAPH.CENTER, space_after=40)

# Info table
meta = [
    ("Version", "1.0"),
    ("Date", "September 2026"),
    ("Event", "Smart India Hackathon 2026 – SIH25017"),
    ("Ministry", "Ministry of Tribal Affairs (MoTA), Government of India"),
    ("Team", "[Your Team Name] – Team ID: [Your Team ID]"),
    ("Scope", "Frontend-only build (React + TypeScript) with mock API layer"),
]

t = doc.add_table(rows=0, cols=2)
t.alignment = WD_TABLE_ALIGNMENT.CENTER
for k, v in meta:
    row = t.add_row().cells
    row[0].text = ''

```

```

row[1].text = ''
r0 = row[0].paragraphs[0].add_run(k)
r0.bold = True
r0.font.size = Pt(11)
r0.font.color.rgb = NAVY
r0.font.name = 'Calibri'
r1 = row[1].paragraphs[0].add_run(v)
r1.font.size = Pt(11)
r1.font.color.rgb = DARK
r1.font.name = 'Calibri'
row[0].width = Inches(1.6)
row[1].width = Inches(4.5)

page_break(doc)

# =====
# TABLE OF CONTENTS
# =====
add_heading(doc, "Table of Contents", level=1, space_before=0)

toc_items = [
    "1. Executive Summary",
    "2. Problem Statement",
    "3. Product Vision",
    "4. Goals & Success Metrics",
    "5. Target Users & Personas",
    "6. User Roles & Permissions",
    "7. Scope – Feature List",
    "8. Detailed Feature Specifications",
    "9. Information Architecture",
    "10. Data Models",
    "11. Mock API Contract",
    "12. Design System",
    "13. Layout Guidelines",
    "14. Non-Functional Requirements",
    "15. Tech Stack",
    "16. Project Structure",
    "17. Build Plan (7 Days)",
    "18. Demo Script",
    "19. Risks & Mitigations",
    "20. Acceptance Criteria",
    "21. Reference Links",
    "22. Appendix – Demo Credentials",
    "23. Appendix – Sample Mock Data Volume",
]
for item in toc_items:
    add_para(doc, item, size=11, space_after=2)

page_break(doc)

# =====
# 1. EXECUTIVE SUMMARY
# =====
add_heading(doc, "1. Executive Summary", level=1, space_before=0)

```

```
add_para(doc, "ST-SETU is a unified digital platform for end-to-end administration of "
            "Ministry of Tribal Affairs (MoTA) scholarship and fellowship schemes for "
            "Scheduled Tribe (ST) students. This PRD covers the frontend implementation "
            _ "
            "the complete user-facing product that will later integrate with a real bac
            kend.")
```

```
add_para(doc, "The frontend will deliver two role-based portals:", bold=True)
add_bullet(doc, "Applicant Portal – for ST students applying to schemes like NFST and NO
S")
add_bullet(doc, "Admin Portal – for MoTA officers to verify, scrutinize, select, and moni
tor")
```

```
add_para(doc, "The frontend will use realistic mock data and a mock API layer, allowing "
            "a fully clickable demo without a live backend. Every screen, workflow, and "
            "
            "state will be functional.")
```

```
add_para(doc, "Primary Goal: Demonstrate a production-grade, transparent, AI-assisted "
            "scholarship management experience that can be handed to a backend team "
            "for integration.", bold=True)
```

```
# =====
# 2. PROBLEM STATEMENT
# =====
```

```
add_heading(doc, "2. Problem Statement", level=1)
```

```
add_para(doc, "The Ministry of Tribal Affairs runs multiple scholarship and fellowship "
            "schemes. Today, the process involves:")
```

```
for item in [
    "Manual scrutiny of thousands of applications",
    "Repeated correspondence with applicants for missing documents",
    "Multiple levels of verification across officers",
    "Limited real-time visibility into scheme progress",
    "No unified platform – each scheme runs on separate workflows",
    "Errors and delays in eligibility verification",
    "No transparency for applicants on their application status",
]:
```

```
    add_bullet(doc, item)
```

```
add_para(doc, "There is a need for a secure, transparent, intelligent digital platform "
            "that streamlines the end-to-end administration of these schemes.",
            italic=True)
```

```
# =====
# 3. PRODUCT VISION
# =====
```

```
add_heading(doc, "3. Product Vision", level=1)
```

```
add_para(doc, "\u201cOne integrated, configurable, AI-assisted platform that takes an ST
"
            "student from application to disbursement – with transparency for the "
            "student and full oversight for the Ministry.\u201d",
```

```

        italic=True, size=12, color=NAVY)

add_para(doc, "The frontend is the entire experience layer of this vision. It must:")
for item in [
    "Feel like a real government digital service – trustworthy, accessible, professional",
    "Support both roles with distinct, optimized interfaces",
    "Surface AI assistance visibly – OCR extraction, confidence scores, deficiency flags, anomaly alerts",
    "Make the workflow transparent – every applicant can see exactly where their application is",
    "Make the admin workflow efficient – bulk actions, filters, dashboards",
]:
    add_numbered(doc, item)

# =====
# 4. GOALS & SUCCESS METRICS
# =====
add_heading(doc, "4. Goals & Success Metrics", level=1)

add_heading(doc, "4.1 Product Goals", level=2)
add_table(doc,
    ["#", "Goal", "How we know it's achieved"],
    [
        ["G1", "Applicant can apply end-to-end in under 15 minutes", "Time-on-task in usability testing"],
        ["G2", "Officer can review and act on an application in under 2 minutes", "Time-on-task with sample data"],
        ["G3", "All workflow states are visible and understandable", "Applicant can articulate status without help"],
        ["G4", "AI assistance is visible but never overrides human judgment", "Officers see confidence scores; all final decisions are human"],
        ["G5", "Works on mobile and desktop", "Responsive at 375px, 768px, 1280px, 1920px"],
    ],
    col_widths=[0.5, 3.0, 3.5])

add_heading(doc, "4.2 Success Metrics (Demo KPIs)", level=2)
for item in [
    "15+ pages fully functional with mock data",
    "2 roles with role-based routing",
    "7-step application form with validation and draft saving",
    "5+ charts on admin dashboard",
    "Lighthouse score \u2265 90 on Performance, Accessibility, Best Practices",
    "Zero console errors on demo path",
]:
    add_bullet(doc, item)

add_heading(doc, "4.3 Out of Scope (for this build)", level=2)
for item in [
    "Real backend / database",
    "Real authentication (mock only)",
    "Real OCR / LLM calls (simulated)",
    "Real file storage (localStorage only)",
    "Payment / disbursement integration",

```

```

    "SMS / Email gateways",
    "Aadhaar / DigiLocker integration",
]:
    add_bullet(doc, item)

# =====
# 5. TARGET USERS & PERSONAS
# =====
add_heading(doc, "5. Target Users & Personas", level=1)

personas = [
    {
        "name": "Persona 1 – Priya, ST Applicant",
        "attrs": [
            ("Age", "23"),
            ("Context", "M.Sc. student in Odisha, applying for NFST fellowship for Ph.
D."),
            ("Tech comfort", "Moderate – uses smartphone daily, but not tech-savvy"),
            ("Needs", "Simple application, clear status, no repeated document uploads"),
            ("Pain points", "Doesn't know why applications get rejected; can't track stat
us; has to call the ministry"),
            ("Success", "Applies once, sees status, gets approved without follow-up"),
        ],
    },
    {
        "name": "Persona 2 – Shri Rajesh, Verification Officer",
        "attrs": [
            ("Age", "45"),
            ("Context", "MoTA officer, handles 200+ applications per day during peak seas
on"),
            ("Tech comfort", "Moderate – uses government portals daily"),
            ("Needs", "Fast verification, clear deficiency marking, bulk actions, audit t
rail"),
            ("Pain points", "Manual data entry from PDFs; can't prioritize applications;
no clear queue"),
            ("Success", "Clears queue efficiently, correct decisions, no rework"),
        ],
    },
    {
        "name": "Persona 3 – Dr. Meera, Selection Committee Member",
        "attrs": [
            ("Age", "52"),
            ("Context", "Academic on MoTA committee, reviews shortlisted applications for
final merit list"),
            ("Tech comfort", "Low-moderate"),
            ("Needs", "Clear merit list, side-by-side comparison, decision rationale"),
            ("Pain points", "No structured view of applicants; committee spends hours on
Excel"),
            ("Success", "Finalizes merit list in one session, defensible decisions"),
        ],
    },
    {
        "name": "Persona 4 – Smt. Kavita, MoTA Administrator",
        "attrs": [
            ("Age", "40"),

```

```

        ("Context", "Oversees all schemes, reports to leadership weekly"),
        ("Tech comfort", "Moderate"),
        ("Needs", "Dashboards, scheme-wise analytics, exportable reports"),
        ("Pain points", "No real-time data; relies on manual reports from officers"),
        ("Success", "Opens dashboard, sees live status, exports report in 2 clicks"),
    ],
},
]
for persona in personas:
    add_heading(doc, persona["name"], level=3)
    for k, v in persona["attrs"]:
        p = doc.add_paragraph()
        p.paragraph_format.left_indent = Inches(0.25)
        p.paragraph_format.space_after = Pt(2)
        r1 = p.add_run(f"{k}: ")
        r1.bold = True
        r1.font.size = Pt(10.5)
        r1.font.color.rgb = NAVY
        r2 = p.add_run(v)
        r2.font.size = Pt(10.5)
        r2.font.color.rgb = DARK

# =====
# 6. USER ROLES & PERMISSIONS
# =====
add_heading(doc, "6. User Roles & Permissions", level=1)

add_table(doc,
    ["Role", "Can access", "Cannot access"],
    [
        ["Guest", "Landing, Login, Register, Scheme info", "Any dashboard, any application"],
        ["Applicant", "Own profile, own applications, browse schemes, apply, view own status, notifications, documents", "Admin portal, other applicants' data"],
        ["Verification Officer", "Admin dashboard, applications list, verification queue, application review, raise deficiency, approve/reject", "Scheme config, final selection, admin settings"],
        ["Scrutiny Officer", "All of the above + scrutiny queue, mark applications for selection", "Final merit list approval"],
        ["Selection Committee", "All of the above + merit list, ranking, final approval", "Scheme config"],
        ["MoTA Admin", "Everything – scheme config, dashboards, reports, communications, settings, audit logs", "-"],
    ],
    col_widths=[1.2, 3.0, 2.5])

add_para(doc, "For the frontend build, we will implement two primary roles (Applicant and "
    "Admin) with the admin role having the full set of admin capabilities. "
    "Sub-roles can be simulated via a role switcher in the demo.",
    italic=True)

# =====
# 7. SCOPE – FEATURE LIST

```

```
# =====
add_heading(doc, "7. Scope – Feature List", level=1)

add_heading(doc, "7.1 Public Pages", level=2)
add_table(doc,
    ["ID", "Feature", "Priority", "Notes"],
    [
        ["PUB-1", "Landing page", "P0", "Hero, schemes overview, CTAs, footer"],
        ["PUB-2", "Login page", "P0", "Role-based redirect after login"],
        ["PUB-3", "Register page", "P0", "Applicant registration with validation"],
        ["PUB-4", "Forgot password", "P1", "Simulated flow"],
        ["PUB-5", "About / Help page", "P2", "Static content"],
    ],
    col_widths=[0.7, 1.8, 0.8, 3.0])

add_heading(doc, "7.2 Applicant Portal", level=2)
add_table(doc,
    ["ID", "Feature", "Priority", "Notes"],
    [
        ["APP-1", "Applicant dashboard", "P0", "KPIs, latest application status, notifications"],
        ["APP-2", "Browse schemes", "P0", "NFST, NOS, others with eligibility summary"],
        ["APP-3", "Scheme detail", "P0", "Full eligibility, documents required, timeline"],
        ["APP-4", "Multi-step apply form", "P0", "7 steps, draft save, validation, scheme-specific fields"],
        ["APP-5", "Document upload", "P0", "Drag-drop, preview, simulate OCR extraction"],
        ["APP-6", "My Applications", "P0", "Table with status chips, filters, search"],
        ["APP-7", "Application detail", "P0", "Full view, status timeline, deficiency panel"],
        ["APP-8", "Respond to deficiency", "P0", "Upload revised documents, resubmit"],
        ["APP-9", "Notifications", "P0", "In-app notification center"],
        ["APP-10", "Profile management", "P1", "Edit profile, change password"],
        ["APP-11", "Download acknowledgment", "P1", "PDF of submitted application"],
    ],
    col_widths=[0.7, 1.8, 0.8, 3.0])

add_heading(doc, "7.3 Admin Portal", level=2)
add_table(doc,
    ["ID", "Feature", "Priority", "Notes"],
    [
        ["ADM-1", "Admin dashboard", "P0", "KPI cards + 5 charts + recent queue"],
        ["ADM-2", "Applications list", "P0", "TanStack table, filters, search, bulk actions"],
        ["ADM-3", "Application review", "P0", "Split view: data + OCR-annotated documents"],
        ["ADM-4", "Verification queue", "P0", "Priority-sorted, confidence scores"],
        ["ADM-5", "Deficiency raise", "P0", "Modal with checklist, sends notification"],
        ["ADM-6", "Scrutiny workflow", "P0", "Mark verified apps for scrutiny"],
        ["ADM-7", "Selection / Merit list", "P0", "Rank, reorder, generate merit list"],
        ["ADM-8", "Scheme config engine", "P0", "Create/edit schemes with dynamic rules"],
        ["ADM-9", "Communications", "P1", "Template list, bulk send, log"],
        ["ADM-10", "Reports & analytics", "P0", "Charts, filters, export"],
        ["ADM-11", "Audit log", "P1", "Every action logged, filterable"],
    ],
    col_widths=[0.7, 1.8, 0.8, 3.0])
```

```

        ["ADM-12", "Settings", "P2", "Roles, permissions, profile"],
    ],
    col_widths=[0.7, 1.8, 0.8, 3.0])

add_heading(doc, "7.4 Shared", level=2)
add_table(doc,
    ["ID", "Feature", "Priority"],
    [
        ["SH-1", "Role-based routing", "P0"],
        ["SH-2", "Protected routes", "P0"],
        ["SH-3", "Toast notifications", "P0"],
        ["SH-4", "Loading states", "P0"],
        ["SH-5", "Empty states", "P0"],
        ["SH-6", "Error states", "P0"],
        ["SH-7", "Responsive layouts", "P0"],
        ["SH-8", "Dark mode (optional)", "P2"],
        ["SH-9", "Multi-language EN/HI (optional)", "P2"],
    ],
    col_widths=[0.8, 3.5, 1.0])

# =====
# 8. DETAILED FEATURE SPECIFICATIONS
# =====
add_heading(doc, "8. Detailed Feature Specifications", level=1)

add_heading(doc, "8.1 Multi-Step Application Form (APP-4)", level=2)
add_para(doc, "Purpose: Allow applicant to complete a full application across multiple "
    "steps with the ability to save and resume.", italic=True)
add_para(doc, "Steps:", bold=True)
for item in [
    "Personal Details – Name, DOB, gender, category (ST auto-selected), Aadhaar (masked), phone, email",
    "Address & Domicile – State, district, PIN, permanent address, domicile certificate reference",
    "Academic Details – Highest qualification, institution, course, year, marks/percentage",
    "Scheme-Specific – NFST: research topic, guide, university, Ph.D. reg. no. / NOS: foreign university, country, program, admission letter reference, GRE/TOEFL scores",
    "Bank Details – Account number, IFSC, bank name, branch, passbook upload",
    "Documents Upload – Caste cert, income cert, marksheets, admission letter, passport (NOS only)",
    "Review & Submit – Read-only summary with edit links, declaration checkbox, submit",
]:
    add_numbered(doc, item)

add_para(doc, "Behavior:", bold=True)
for item in [
    "Progress stepper visible at top",
    "Save as Draft at every step (auto-saves to localStorage on change)",
    "Validation with react-hook-form + zod, per step",
    "Cannot proceed if current step has errors",
    "On submit, generate application ID like NFST/2026/00123",
    "Confirmation screen with download acknowledgment PDF",
]:
    add_bullet(doc, item)

```

```

add_para(doc, "Acceptance Criteria:", bold=True)
for item in [
    "Draft saved and restored across browser refresh",
    "All fields validated with India-specific rules (PIN code 6 digits, IFSC format, Aadhaar 12 digits, phone 10 digits)",
    "Scheme-specific fields render correctly based on schemeId param",
    "Review step shows all entered data in read-only cards",
]:
    add_bullet(doc, item)

add_heading(doc, "8.2 Application Review with OCR Preview (ADM-3)", level=2)
add_para(doc, "Purpose: Allow officer to verify an application against uploaded documents"
        "with AI-extracted data visible.", italic=True)

add_para(doc, "Layout:", bold=True)
add_bullet(doc, "Left panel (40%): Applicant data in read-only cards – Personal, Address, Academic, Bank, Scheme-specific. Each field has a small \u201cverified\u201d checkbox.")
add_bullet(doc, "Right panel (60%): Document viewer – tabs for each uploaded document, PDF/image preview, overlay of extracted fields with confidence bars, low-confidence (<70%) fields highlighted red, click a field to jump to source.")

add_para(doc, "Actions (bottom bar):", bold=True)
add_bullet(doc, "\u2705 Approve – marks application as verified")
add_bullet(doc, "\u274c Reject – opens modal with reason (predefined list + free text)")
add_bullet(doc, "\u26a0\u207f Raise Deficiency – opens modal with checklist: missing document, illegible document, data mismatch, expired certificate, other (free text). Sends notification to applicant.")

add_para(doc, "Acceptance Criteria:", bold=True)
for item in [
    "Officer can see all data and documents on one screen",
    "Every action logs to audit trail",
    "Deficiency modal sends a mock notification visible in applicant portal",
    "Confidence scores visible on extracted fields",
    "Keyboard shortcuts: A = approve, R = reject, D = deficiency",
]:
    add_bullet(doc, item)

add_heading(doc, "8.3 Admin Dashboard (ADM-1)", level=2)
add_para(doc, "Purpose: Give MoTA leadership a real-time view of all scheme activity.", italic=True)

add_para(doc, "KPI Cards (top row):", bold=True)
for item in ["Total Applications (with % change)", "Pending Verification", "Verified", "Selected", "Rejected / Deficient"]:
    add_bullet(doc, item)

add_para(doc, "Charts (middle rows):", bold=True)
for item in [
    "Line chart: Applications submitted over last 30 days",
    "Donut chart: Scheme-wise split (NFST vs NOS vs others)",
    "Bar chart: State-wise applications (top 10 states)",

```

```

        "Funnel chart: Submitted \u2192 Verified \u2192 Scrutinized \u2192 Selected \u2192 Disbursed",
        "Stacked bar: Deficiency reasons breakdown",
    ]:
        add_bullet(doc, item)

add_para(doc, "Recent Activity:", bold=True)
add_bullet(doc, "Table of last 10 submissions needing action")
add_bullet(doc, "Anomaly alert panel (AI-flagged applications)")

add_para(doc, "Filters: Date range, Scheme, State, Status", bold=True)

add_heading(doc, "8.4 Scheme Configuration Engine (ADM-8)", level=2)
add_para(doc, "Purpose: Allow MoTA admin to configure new schemes without code changes.",
          italic=True)

add_para(doc, "Fields:", bold=True)
for item in [
    "Scheme name, code, description",
    "Application window (start/end dates)",
    "Eligibility rules (dynamic list): Field, Operator (eq/lt/gt/in/range), Value, Logic (AND/OR)",
    "Required documents (checklist with add/remove)",
    "Workflow stages (drag-to-reorder list)",
    "Selection criteria (merit vs need-based, weights)",
]:
    add_bullet(doc, item)

add_heading(doc, "8.5 Deficiency Workflow (APP-8 + ADM-5)", level=2)
add_para(doc, "Flow:", bold=True)
for item in [
    "Officer raises deficiency \u2192 status changes to query_raised",
    "Applicant sees notification + red badge on application",
    "Applicant opens application \u2192 sees deficiency card with list of issues",
    "Applicant uploads corrected documents \u2192 clicks Resubmit",
    "Status changes back to under_verification",
    "Officer sees resubmitted application in queue with \u201cResubmission\u201d badge",
]:
    add_numbered(doc, item)

# =====
# 9. INFORMATION ARCHITECTURE
# =====

add_heading(doc, "9. Information Architecture", level=1)
ia = """ST-SETU
|
|─ Public
|   |─ Landing (/)
|   |─ Login (/login)
|   |─ Register (/register)
|   |─ Help (/help)
|
|─ Applicant Portal (/app)

```

```

|   ├── Dashboard
|   ├── Schemes
|   |   ├── List
|   |   └── Detail (:schemeId)
|   ├── Apply (:schemeId)      [multi-step form]
|   ├── My Applications
|   |   ├── List
|   |   └── Detail (:applicationId)
|   ├── Documents
|   ├── Notifications
|   └── Profile
|
└── Admin Portal (/admin)
    ├── Dashboard
    ├── Applications
    |   ├── List
    |   └── Review (:applicationId)
    ├── Verification Queue
    ├── Scrutiny
    ├── Selection
    |   └── Merit List
    ├── Schemes
    |   ├── List
    |   └── Configure (:schemeId)
    ├── Communications
    ├── Reports
    ├── Audit Log
    └── Settings""

```

```

└── Admin Portal (/admin)
    ├── Dashboard
    ├── Applications

```

```

    |   ├── List
    |   └── Review (:applicationId)
    ├── Verification Queue
    ├── Scrutiny
    ├── Selection
    |   └── Merit List
    ├── Schemes
    |   ├── List
    |   └── Configure (:schemeId)
    ├── Communications
    ├── Reports
    ├── Audit Log
    └── Settings""

```

```
add_code_block(doc, ia)
```

```
# =====
```

```
# 10. DATA MODELS
```

```
# =====
```

```
add_heading(doc, "10. Data Models (TypeScript Interfaces)", level=1)
```

```
types_code = """export type Role = 'applicant' | 'admin' | 'officer' | 'committee';
```

```
export type ApplicationStatus =
```

```

| 'draft'
| 'submitted'
| 'under_verification'
| 'query_raised'
| 'verified'
| 'scrutinized'
| 'selected'
| 'rejected'
| 'disbursed';

```

```
export interface User {
```

```

  id: string;
  name: string;
  email: string;
  phone: string;
  role: Role;
  createdAt: string;

```

```

}

export interface Scheme {
  id: string;
  code: 'NFST' | 'NOS' | string;
  name: string;
  description: string;
  window: { start: string; end: string };
  eligibility: EligibilityRule[];
  requiredDocs: string[];
  stages: string[];
  selectionCriteria: 'merit' | 'need' | 'hybrid';
}

```

```

export interface EligibilityRule {
  field: string;
  operator: 'eq' | 'lt' | 'gt' | 'in' | 'range';
  value: string | number | string[];
  logic: 'AND' | 'OR';
}

```

```

export interface Application {
  id: string;
  applicantId: string;
  applicantName: string;
  schemeId: string;
  schemeCode: string;
  status: ApplicationStatus;
  currentStage: number;
  submittedAt: string;
  lastUpdatedAt: string;
  personal: PersonalDetails;
  address: AddressDetails;
  academic: AcademicDetails;
  schemeSpecific: Record<string, any>;
  bank: BankDetails;
  documents: Document[];
  ocrFields?: OCRField[];
  deficiency?: Deficiency;
  score?: number;
  remarks?: string;
}

```

```

export interface Document {
  id: string;
  type: string;
  fileName: string;
  uploadedAt: string;
  status: 'pending' | 'verified' | 'deficient';
  url: string;
}

```

```

export interface OCRField {
  field: string;
  value: string;
  confidence: number; // 0-100
}

```

```

    sourceDocId: string;
    sourcePage: number;
}

export interface Deficiency {
    raisedAt: string;
    raisedBy: string;
    reasons: string[];
    note?: string;
    resolvedAt?: string;
    round: number;
}

export interface Notification {
    id: string;
    userId: string;
    title: string;
    message: string;
    type: 'info' | 'success' | 'warning' | 'error';
    read: boolean;
    link?: string;
    createdAt: string;
}

export interface AuditLogEntry {
    id: string;
    actorId: string;
    actorName: string;
    action: string;
    entityType: 'application' | 'scheme' | 'user';
    entityId: string;
    timestamp: string;
    metadata?: Record<string, any>;
}"""
add_code_block(doc, types_code)

# =====
# 11. MOCK API CONTRACT
# =====

add_heading(doc, "11. Mock API Contract", level=1)
add_para(doc, "All frontend pages consume this interface. Later, replace with real fetch
() "
        "calls – no UI rewrites needed.", italic=True)

api_code = """export const mockApi = {
    // Auth
    login(email: string, password: string): Promise<{ user: User; token: string }>;
    register(data: RegisterInput): Promise<User>;
    logout(): Promise<void>;

    // Schemes
    getSchemes(): Promise<Scheme[]>;
    getSchemeById(id: string): Promise<Scheme>;
    createScheme(data: Partial<Scheme>): Promise<Scheme>;
    updateScheme(id: string, data: Partial<Scheme>): Promise<Scheme>;

```

```

// Applications
getApplications(filters?: ApplicationFilters): Promise<Application[]>;
getApplicationById(id: string): Promise<Application>;
getMyApplications(userId: string): Promise<Application[]>;
submitApplication(data: Partial<Application>): Promise<Application>;
updateApplication(id: string, data: Partial<Application>): Promise<Application>;

// Documents
uploadDocument(file: File, applicationId: string): Promise<Document>;
runOCR(documentId: string): Promise<OCRField[]>;

// Workflow
approveApplication(id: string): Promise<Application>;
rejectApplication(id: string, reason: string): Promise<Application>;
raiseDeficiency(id: string, reasons: string[], note?: string): Promise<Application>;
resubmitApplication(id: string, documents: File[]): Promise<Application>;
markForSelection(id: string, score: number): Promise<Application>;
generateMeritList(schemeId: string): Promise<Application[]>;

// Dashboards
getAdminStats(filters?: StatsFilters): Promise<AdminStats>;
getApplicantStats(userId: string): Promise<ApplicantStats>;

// Notifications
getNotifications(userId: string): Promise<Notification[]>;
markNotificationRead(id: string): Promise<void>;

// Audit
getAuditLog(filters?: AuditFilters): Promise<AuditLogEntry[]>;
}; ""
add_code_block(doc, api_code)

```

```

# =====
# 12. DESIGN SYSTEM
# =====
add_heading(doc, "12. Design System", level=1)

add_heading(doc, "12.1 Color Palette", level=2)
add_table(doc,
  ["Token", "Hex", "Usage"],
  [
    ["navy", "#1F3864", "Headers, primary text, primary buttons"],
    ["navy-light", "#2E508C", "Hover states"],
    ["orange", "#ED7D31", "Accent, CTAs, active states, highlights"],
    ["green", "#4CAF50", "Success, verified, selected"],
    ["red", "#C62828", "Errors, rejected, deficient"],
    ["teal", "#2A9D8F", "Info, secondary accents"],
    ["purple", "#6F42A0", "Selection, committee"],
    ["yellow", "#F5C218", "Warnings, pending"],
    ["bg", "#F8FAFC", "Page background"],
    ["card", "#FFFFFF", "Cards, panels"],
    ["border", "#E2E8F0", "Dividers, borders"],
    ["text-primary", "#1A202C", "Body text"],
    ["text-muted", "#718096", "Secondary text"],

```

```

    ],
    col_widths=[1.3, 1.0, 3.5])

add_heading(doc, "12.2 Typography", level=2)
add_table(doc,
    ["Style", "Font", "Size", "Weight"],
    [
        ["Display", "Inter", "32px", "700"],
        ["H1", "Inter", "24px", "700"],
        ["H2", "Inter", "20px", "600"],
        ["H3", "Inter", "18px", "600"],
        ["Body", "Inter", "14px", "400"],
        ["Small", "Inter", "12px", "400"],
        ["Mono", "JetBrains Mono", "13px", "400"],
    ],
    col_widths=[1.3, 1.5, 1.0, 1.0])

add_heading(doc, "12.3 Spacing", level=2)
add_para(doc, "Scale: 4 / 8 / 12 / 16 / 20 / 24 / 32 / 40 / 48 / 64")

add_heading(doc, "12.4 Components", level=2)
for item in [
    "Buttons: primary (navy), secondary (outline), danger (red), ghost",
    "Cards: 12px radius, shadow-sm, hover shadow-md",
    "Chips: status badges with role-appropriate colors",
    "Inputs: 8px radius, 1px border, focus ring in orange",
    "Stepper: horizontal with numbered circles",
    "Timeline: vertical with connecting line",
    "Table: zebra rows, sticky header, sortable columns",
    "Modal: centered, backdrop blur, close on Esc",
    "Toast: top-right, 4s auto-dismiss, stackable",
]:
    add_bullet(doc, item)

# =====
# 13. LAYOUT GUIDELINES
# =====
add_heading(doc, "13. Layout Guidelines", level=1)

add_heading(doc, "Applicant Portal", level=3)
for item in [
    "Desktop: Sidebar (240px) + main content (max-width 1200px)",
    "Tablet: Collapsible sidebar",
    "Mobile: Bottom nav with 5 items (Dashboard, Schemes, Apply, Applications, Profile)",
]:
    add_bullet(doc, item)

add_heading(doc, "Admin Portal", level=3)
for item in [
    "Desktop: Sidebar (260px) + main content (max-width 1440px) – wider for tables",
    "Tablet: Collapsible sidebar",
    "Mobile: Not primary use case – show \u201cBest viewed on desktop\u201d notice",
]:
    add_bullet(doc, item)

```

```

add_heading(doc, "Header", level=3)
for item in [
    "Navy background bar",
    "Page title on left",
    "Notifications bell, profile menu on right",
]:
    add_bullet(doc, item)

# =====
# 14. NON-FUNCTIONAL REQUIREMENTS
# =====
add_heading(doc, "14. Non-Functional Requirements", level=1)

add_heading(doc, "Performance", level=3)
for item in ["First Contentful Paint < 1.5s", "Time to Interactive < 3s on 4G",
    "Lighthouse Performance \u2265 90", "Bundle size < 500 KB gzipped"]:
    add_bullet(doc, item)

add_heading(doc, "Accessibility", level=3)
for item in ["WCAG 2.1 AA compliant", "Keyboard navigable", "Screen reader tested",
    "Color contrast \u2265 4.5:1", "Focus indicators visible",
    "ARIA labels on all interactive elements"]:
    add_bullet(doc, item)

add_heading(doc, "Responsiveness", level=3)
for item in ["Breakpoints: 375px, 640px, 768px, 1024px, 1280px, 1536px",
    "Mobile-first CSS", "Touch targets \u2265 44px"]:
    add_bullet(doc, item)

add_heading(doc, "Browser Support", level=3)
add_para(doc, "Chrome 90+, Edge 90+, Firefox 88+, Safari 14+. No IE support.")

add_heading(doc, "Internationalization (Optional)", level=3)
add_para(doc, "English (default), Hindi (i18n-ready with react-i18next)")

add_heading(doc, "Security (Frontend-Level)", level=3)
for item in ["No sensitive data in localStorage beyond mock scope",
    "Role-based route guards", "Input sanitization on all forms",
    "No dangerouslySetInnerHTML"]:
    add_bullet(doc, item)

# =====
# 15. TECH STACK
# =====
add_heading(doc, "15. Tech Stack", level=1)
add_table(doc,
    ["Layer", "Choice"],
    [
        ["Framework", "React 19 + Vite"],
        ["Language", "TypeScript 5"],
        ["Styling", "Tailwind CSS 4"],
        ["Components", "shadcn/ui + Radix UI"],
        ["Icons", "lucide-react"],
        ["Routing", "react-router-dom v7"],
    ],

```

```

    ["State", "Zustand"],
    ["Forms", "react-hook-form + zod"],
    ["Tables", "TanStack Table v8"],
    ["Charts", "Recharts"],
    ["Toasts", "sonner"],
    ["File upload", "react-dropzone"],
    ["PDF preview", "react-pdf"],
    ["Dates", "date-fns"],
    ["Utils", "clsx + tailwind-merge"],
    ["Mock API", "MSW or plain async functions"],
    ["Persistence", "localStorage"],
    ["Deployment", "Vercel"],
  ],
  col_widths=[1.8, 4.0])

```

```
# =====
```

```
# 16. PROJECT STRUCTURE
```

```
# =====
```

```
add_heading(doc, "16. Project Structure", level=1)
```

```
struct = """st-setu-frontend/
```

```
├─ public/
│   └─ mock/
├─ src/
│   ├─ components/
│   │   ├─ ui/           # shadcn primitives
│   │   ├─ layout/       # Navbar, Sidebar, Footer, PageHeader
│   │   ├─ applicant/    # Applicant-specific components
│   │   ├─ admin/        # Admin-specific components
│   │   └─ shared/        # DashboardCard, ChartCard, Stepper, EmptyState
│   ├─ pages/
│   │   ├─ Landing.tsx
│   │   ├─ auth/
│   │   ├─ applicant/
│   │   ├─ admin/
│   │   └─ NotFound.tsx
│   ├─ context/
│   │   └─ AuthContext.tsx
│   ├─ hooks/
│   ├─ lib/
│   │   ├─ mockApi.ts
│   │   ├─ mockData.ts
│   │   ├─ utils.ts
│   │   └─ validators.ts
│   ├─ routes/
│   │   └─ AppRoutes.tsx
│   ├─ styles/
│   ├─ types/
│   │   └─ index.ts
│   ├─ App.tsx
│   └─ main.tsx
├─ .antigravity/
│   └─ rules
├─ .env
├─ index.html
└─ package.json

```

```
|─ tailwind.config.ts
|─ tsconfig.json
└─ README.md""
add_code_block(doc, struct)
```

```
# =====
# 17. BUILD PLAN
# =====
add_heading(doc, "17. Build Plan (7 Days)", level=1)
add_table(doc,
    ["Day", "Deliverable"],
    [
        ["Day 1", "Project setup, .antigravity/rules, design tokens, layout shell (Navba
r, Sidebar, Footer)"],
        ["Day 2", "Auth pages, AuthContext, role guards, routing, mock data + mock API ba
se"],
        ["Day 3", "Applicant dashboard, Browse schemes, Scheme detail, My Applications, A
pplication detail"],
        ["Day 4", "Multi-step apply form, Document upload, Deficiency response"],
        ["Day 5", "Admin dashboard with charts, Applications table, Application review wi
th OCR preview"],
        ["Day 6", "Verification queue, Scrutiny, Selection, Merit list, Scheme config, Co
mmunications"],
        ["Day 7", "Polish, animations, responsive fixes, accessibility pass, deploy, demo
video"],
    ],
    col_widths=[0.8, 5.5])
```

```
# =====
# 18. DEMO SCRIPT
# =====
add_heading(doc, "18. Demo Script (for SIH judges)", level=1)
add_para(doc, "Total: 6 minutes", bold=True)
for item in [
    "0:00\u2013 Landing page. Introduce problem and ST-SETU.",
    "0:30\u2013 Register as applicant. Show validation.",
    "1:00\u2013 Browse schemes \u2013 Apply for NFST \u2013 Complete 7-step form \u2013
2192 Upload docs \u2013 Submit. Show draft auto-save.",
    "2:30\u2013 Applicant dashboard. Show status timeline and notification.",
    "3:00\u2013 Switch to admin (demo credentials). Show dashboard with live KPIs a
nd charts.",
    "3:30\u2013 Open the same application in review. Show OCR-extracted fields with
confidence scores. Raise a deficiency.",
    "4:30\u2013 Switch back to applicant. Show notification and deficiency response
flow.",
    "5:00\u2013 Back to admin. Approve. Show status change.",
    "5:30\u2013 Merit list generation. Reports. Wrap up with impact metrics.",
]:
    add_bullet(doc, item)
```

```
# =====
# 19. RISKS & MITIGATIONS
# =====
```

```

add_heading(doc, "19. Risks & Mitigations", level=1)
add_table(doc,
    ["Risk", "Impact", "Mitigation"],
    [
        ["Scope creep", "Missing demo deadline", "Strict P0/P1/P2 prioritization. P2 features dropped if needed."],
        ["Mock data too thin", "Demo feels hollow", "Generate 50+ realistic applications with Indian names, states, institutions"],
        ["OCR simulation unconvincing", "AI value unclear", "Pre-compute mock OCR results with realistic confidence scores"],
        ["Mobile view broken", "Accessibility concerns", "Test at 375px on every page"],
        ["Theme inconsistency", "Looks unprofessional", "Design tokens in Tailwind config; no hardcoded colors"],
        ["Role confusion", "Demo breaks", "Clear role switcher in dev mode; test both paths daily"],
        ["Slow build due to agent loops", "Missing features", "Use Antigravity /planning before each major feature; /fork for parallel work"],
    ],
    col_widths=[1.7, 1.8, 3.5])

# =====
# 20. ACCEPTANCE CRITERIA
# =====
add_heading(doc, "20. Acceptance Criteria (Definition of Done)", level=1)
add_para(doc, "The frontend is done when:", bold=True)
for item in [
    "All P0 features are implemented and functional",
    "Both roles (applicant, admin) can complete their full workflow end-to-end",
    "Application state persists across browser refresh (localStorage)",
    "All screens are responsive at 375px, 768px, 1280px",
    "Lighthouse \u2265 90 on Performance, Accessibility, Best Practices",
    "Zero console errors on demo path",
    "All forms validated with India-specific rules",
    "All charts render with mock data and filters work",
    "Deficiency workflow works end-to-end (raise \u2192 notify \u2192 respond \u2192 re-verify)",
    "Merit list can be generated and exported",
    "Deployed to Vercel with a public URL",
    "Demo video recorded (\u2264 6 minutes)",
    "README includes setup, credentials, and architecture",
    ".antigravity/rules documented and shared with team",
]:
    p = doc.add_paragraph(style='List Bullet')
    p.paragraph_format.left_indent = Inches(0.3)
    p.paragraph_format.space_after = Pt(3)
    run = p.add_run("\u2610 " + item)
    run.font.size = Pt(11)
    run.font.name = 'Calibri'

# =====
# 21. REFERENCE LINKS
# =====
add_heading(doc, "21. Reference Links", level=1)
refs = [

```

```

("MoTA Scholarships", "https://tribal.nic.in/ScholarshiP.aspx"),
("DBT Tribal", "https://dbttribal.gov.in/AllScheme.aspx"),
("National Scholarship Portal", "https://scholarships.gov.in"),
("Digital India", "https://digitalindia.gov.in"),
("WCAG 2.1 Quick Reference", "https://www.w3.org/WAI/WCAG21/quickref/"),
("React 19 Docs", "https://react.dev"),
("Tailwind CSS", "https://tailwindcss.com"),
("shadcn/ui", "https://ui.shadcn.com"),
("Recharts", "https://recharts.org"),
("TanStack Table", "https://tanstack.com/table"),
]
for name, url in refs:
    p = doc.add_paragraph()
    p.paragraph_format.left_indent = Inches(0.3)
    p.paragraph_format.space_after = Pt(3)
    r1 = p.add_run("\u2022 " + name + ": ")
    r1.font.size = Pt(11)
    r1.bold = True
    r1.font.color.rgb = NAVY
    r2 = p.add_run(url)
    r2.font.size = Pt(11)
    r2.font.color.rgb = RGBColor(0x1F, 0x38, 0x64)
    r2.underline = True

# =====
# 22. APPENDIX – DEMO CREDENTIALS
# =====
add_heading(doc, "22. Appendix – Demo Credentials", level=1)
add_table(doc,
    ["Role", "Email", "Password"],
    [
        ["Applicant", "student@demo.in", "demo123"],
        ["Officer", "officer@demo.in", "demo123"],
        ["Admin", "admin@demo.in", "demo123"],
    ],
    col_widths=[1.5, 3.0, 1.5])

# =====
# 23. APPENDIX – SAMPLE MOCK DATA VOLUME
# =====
add_heading(doc, "23. Appendix – Sample Mock Data Volume", level=1)
for item in [
    "3 schemes: NFST, NOS, and one more (e.g., Top Class Education)",
    "50 applicants with realistic Indian names, states, universities",
    "80 applications distributed across schemes and statuses:",
]:
    add_bullet(doc, item)
for sub in [
    "20 submitted", "15 under verification", "10 query raised",
    "15 verified", "10 scrutinized", "8 selected", "2 rejected",
]:
    add_bullet(doc, sub, level=1)
add_bullet(doc, "200+ documents across applications with mock OCR results")
add_bullet(doc, "10 notifications per applicant on average")

```

```

add_bullet(doc, "300+ audit log entries")

# ----- END
add_para(doc, "\n\n\u2014\u2014\u2014 END OF DOCUMENT \u2014\u2014\u2014",
        size=10, italic=True, color=GRAY,
        align=WD_ALIGN_PARAGRAPH.CENTER, space_after=0)

# ----- SAVE
output = "ST-SETU-PRD.docx"
doc.save(output)
print(f"\u2714 Generated: {output}")
print("Open it in Word / Google Docs / LibreOffice to review and edit.")

```

What the generated document contains

- **Title page** with project name, tagline, event info, and meta table
- **Table of Contents** (23 sections, manual list — you can convert to auto-TOC in Word)
- **All 23 sections** from the PRD in clean, formatted text
- **10+ styled tables** (personas, roles, features, design tokens, tech stack, risks, credentials, etc.)
- **Code blocks** with light-gray shading for TypeScript interfaces, API contract, project structure
- **Navy headings + orange accents** matching the SIH theme
- **Page breaks** between major sections

After generating

1. Open `ST-SETU-PRD.docx` in **Word** or **Google Docs**
2. Update the `[Your Team Name]` and `[Your Team ID]` on the title page
3. Optionally: **References** → **Table of Contents** to convert the TOC to an auto-updating one
4. **File** → **Save As** → **PDF** if you need a PDF version to submit alongside your SIH PPT
5. Share the `.docx` with your team via Google Drive for collaborative edits

Optional — convert to PDF from terminal

If you want a PDF directly:

```
bash
```

```
# Linux / macOS
libreoffice --headless --convert-to pdf ST-SETU-PRD.docx

# or with pandoc
pandoc ST-SETU-PRD.docx -o ST-SETU-PRD.pdf
```

Would you like me to also generate:

1. **A one-page PRD summary** (for stakeholders who won't read the full doc)
2. **The .antigravity/rules file** based on this PRD
3. **The mock data JSON files** ready to drop into your project

Tell me which one and I'll produce it.